Snipesec
CYBER SECURITY AGENCY

PHP2Shell

Teknik Analiz ve Kurumsal Savunma Raporu

Zafiyet incelemesi ve uygulamalı SnipeLab doğrulaması

CVE-2026-109618 ve CVE-2026-109619

Kamu ve özel sektör BT ekipleri için

PHP çekirdeğindeki bellek güvenliği sorunlarının teknik nedenleri, etkilenen çalışma ortamları ve kurumsal çözüm adımları; izole laboratuvar kaynakları ve doğrulama ölçütleriyle birlikte ele alınır.

Teknik mekanizma ve neden–sonuç analizi

Yama, yapılandırma, tespit ve olay müdahalesi

Tam kaynak kodlu SnipeLab laboratuvarı

SNIPELAB

Ömer TOKMAK
Atilla Yusuf Er
Prof. Dr. Arif SARI

Teknik bilgi kesimi 25 Ekim 2026 | Rapor sürümü 3.0
Kurumsal yayın 26 Ekim 2026

github.com/kosovalifaruk/snipelab-php2shell

İçindekiler

BÖLÜM I TEKNİK ANALİZ VE TEHDİDİN KAPSAMI

1. Yönetici Özeti 3
2. Zafiyetin Doğası - Neden ve Nasıl Oluşuyor? 5
3. Keşif süreci ve tehdit görünümü 7
4. Etkilenen Ortamlar ve Envanter Kontrolü 8

BÖLÜM II ÇÖZÜM VE KURUMSAL SAVUNMA

5. Kurumların Yapması Gerekenler 10
6. Tespit ve Olay İncelemesi 13
7. Denetim Kontrol Listesi 14
8. Kurumsal İletişim ve Farkındalık 14

BÖLÜM III UYGULAMALI SNİPELAB LABORATUVARI

9. SnipeLab doğrulama kapsamı ve kanıt sınırı 15
10. Ortamı hazırlayın 15
11. İmajları alın ve kimliklerini kaydedin 16
12. Üç Apache ortamını başlatın 16
13. Gerçek web ortamını kontrol edin 16
14. Otomatik savunma kontrollerini çalıştırın 17
15. İsteğe bağlı: sınırlı OOM ve günlük gözlemi 17
16. İsteğe bağlı: PHP-FPM karşılaştırması 17
17. Tespit sistemini ayrı doğrulayın 18
18. Operatör doğrulama ve kabul kartı 18
19. Kapatma ve temizleme 19

BÖLÜM IV KABUL KANITLARI VE TEKNİK REFERANSLAR

20. Doğrulama durumu ve teknik sınırlılıklar 20
21. Kurumsal kapanış kaydı örneği 20
22. Terimler ve rapor okuma notları 21
23. Kaynaklar 21

EK A SNİPELAB ÇALIŞMA DOSYALARININ TAM KAYNAĞI

24. Paket yerleşimi ve kaynak eşlemesi 23
25. Paket sürümleme ve kurumsal dağıtım notu 30

Amaç kapsam ve hedef kitle

Bu teknik rapor kamu kurumları ve özel sektörün BT, sistem yönetimi, uygulama geliştirme, DevSecOps, SOC ve olay müdahale ekipleri için hazırlanmıştır. PHP2Shell'in kök nedenini, erişilebilirlik koşullarını ve olası etkilerini açıklarken, düzeltmenin çalışan sistem üzerinde nasıl doğrulanacağını tanımlar.

Rapor; teknik analiz, kurumsal müdahale planı ve SnipeLab uygulamalarını bir araya getirir. Laboratuvarın tam çalışma dosyaları Ek A'da ve `php2shell-lab/` paketinde sunulur. Sürüm, SAPI, etkin yapılandırma ve normal yükleme kontrolleri otomatik yürütülür. Sabit boyutlu OOM gözlemi ve soyut yaşam döngüsü modeli ayrı kanıt sınıflarıdır.

Kanıt yaklaşımı: Üretici bülteniyle doğrulanan zafiyet bilgisi, paket üzerinde ölçülen kontrol sonucu ve operatörün gerçek PHP ortamında alacağı deney kaydı ayrı raporlanır. Bu sürümün hazırlama ve runtime doğrulama durumu Bölüm 20'de açıkça verilmiştir. Kontrol paketinin PASS vermesi bütün CVE yollarının sömürü testiyle kapatıldığını göstermez.

İçerik: PDF ve Markdown teknik rapor; Compose, PHP uygulaması, INI/Nginx yapılandırması, check.py, kanıt toplama ve digest sabitleme aracı, yaşam döngüsü modeli, sözleşme testleri ve kullanım yönergesi. Kod dosyaları rapordaki yönergelerle aynı sürümdedir.

Kaynak numaraları bütün raporda tek kaynakçaya bağlanır. Düzeltme kararında PHP ve paket sağlayıcısının güncel duyuruları esas alınır; araştırmacı gözlemleri kaynaklarına atfedilir.

BÖLÜM I TEKNİK ANALİZ VE TEHDİDİN KAPSAMI

1. Yönetici Özeti

PHP2Shell, PHP'nin istek işlerken oluşan ağır hatalardan güvenli biçimde çıkamamasıyla ilişkili bellek güvenliği (memory safety) sorunlarının ortak adıdır. Bazı durumlarda yarım kalmış bir iç yapı geçerli sanılır; bazılarında önceki isteğin belleğine ait işaretçiler sonraki işleme taşınır. Sonuç, hizmet kesintisi, süreç belleğinin açığa çıkması veya uzaktan kod çalıştırma (Remote Code Execution, RCE) olabilir. Bazı yollar oturum açmayı gerektirmez ve uygulama kodu çalışmaya başlamadan erişilebilir. [1-4]

Web üzerinden PHP çalıştıran kurumlar, barındırma hizmeti satın aldıkları sistemler dahil, çalışma ortamlarını envantere almalıdır. Öncelik; internetten erişilebilen hizmetler, hassas veri işleyen uygulamalar ve düzeltme durumu bilinmeyen kurulumlardır. **İnternete açık, yaması doğrulanmamış PHP hizmetlerini kritik öncelikli iş olarak ele alın.** Yalnızca WordPress, tema veya eklenti güncellemek, PHP çalışma ortamının güncellendiğini göstermez.

İlk 24 saat için önerilen kurum akışı:

1. Web siteleri, uygulama programlama arayüzleri (Application Programming Interfaces, API), paneller ve konteynerlerin (containers) PHP sürümü, çalışma biçimi ve sorumlusunu çıkarın.
2. Sağlayıcının güvenlik güncellemesini uygulayın, eski işçi süreçlerini (worker processes) yenileyin ve çalışan web ortamını doğrulayın.
3. Güncelleme bekleyen hizmetlerde resmî geçici önlemleri değerlendirin; aynı zamanda PHP, güvenlik cihazı ve süreç kayıtlarını inceleyin.

Kritik bulgular
Önem: Ortak Zafiyet Puanlama Sistemi (Common Vulnerability Scoring System, CVSS) v4.0 puanı 9.2.
Erişim: Bazı yollar kimlik doğrulaması öncesinde (pre-authentication) erişilebilir; koşullar yola göre değişir.
Kapsam: PHP 5.4'e uzanan aile; her dal ve sağlayıcı paketi ayrıca kontrol edilir.
İlk yamalı kararlı sürümler: 8.2.35, 8.3.36, 8.4.27, 8.5.12.

Ek uyarı: 23 Ekim PHP açıklaması, özel oturum işleyicisi değişiklikleri için ek azaltım önerir; ayrıntı 5.2'dedir. [1-4]

1.1 Bu zafiyet ailesi neden kritik

Riskin kaynağı yalnızca bir sitenin yazdığı PHP dosyaları değildir. Çalışma ortamının kendisi hata durumunu güvenli yönetemiyorsa, iyi yazılmış uygulama kodu bu kusuru ortadan kaldırmaz. Wiz'in gösterimleri, o tarihteki resmî WordPress, Drupal ve Joomla imajlarında kimlik doğrulamasız kod çalıştırmayı ve ayrıca PHP işçi belleğinin okunmasını kapsar. Bu gösterimler örnek hedeflerdir; kapsamı bu üç CMS ile sınırlamaz. [4]

Anonim bir istek saldırı zincirini başlatabilir. Ancak "tek bir istek her kurulumda kesin RCE üretir" sonucu çıkarılmamalıdır: bellek yerleşimi, hata yolu ve işçinin yeniden kullanımı etkili olabilir. mod_php'de uygulama kodundan önce erişilen yol, yalnız uygulama giriş kontrolüne güvenilemeyeceğini gösterir. [1-4]

Kod çalıştırılmasa bile süreç belleğine erişim hassas bilgi açığa çıkarabilir. Bu nedenle değerlendirilecek sonuçlar yalnız web sayfasının değiştirilmesi değildir; oturum ve kimlik bilgilerinin korunması da olay incelemesinin parçasıdır. Heartbleed ve Log4Shell benzetmesi etkiyi tartışmak için kullanılabilir; açıkların teknik olarak aynı olduğu anlamına gelmez. [4]

Teknik neden-sonuç: Çekirdek kusur → uygulamadan bağımsız maruziyet → yalnız CMS güncellemesiyle kapanmayan risk → PHP çalışma ortamını yamalama ve çalışan işçide doğrulama gereği.

1.2 Teknik inceleme kapsamı ve doğrulama yaklaşımı

İnceleme konusu	Uygulanan yaklaşım	**Kanıt veya sınır**
Kaynak kod erişimi	Tam runtime kaynakları Ek A'ya ve pakete eklendi.	Dosya içerikleri ve SHA256SUMS eşlemesi
Kontrol sonucunun kapsamı	Sonuç sınıfları ve JSON kapsam alanları ayrıldı.	cve_exploitation_tested=false
İmaj kimliği ve tekrarlanabilirlik	Gerçek RepoDigest'ten images.lock.env üretiliyor.	Digest değerleri yerel imaj kayıtlarından alınır.
Olumsuz kontrol senaryoları	Olumlu ve olumsuz HTTP sözleşme testleri eklendi.	Gerçekte yürütülen 8 test; PHP yerine geçmez.
Operasyonel kabul ölçütü	Çalışan runtime, modül, işçi, uygulama ve olay kaydı birlikte aranıyor.	Bölüm 5.7 ve 21
Ağ erişimi ve kaynak sınırları	Loopback, internal ağ ve kaynak sınırları kodda tanımlandı.	Yapısal olarak kontrol edildi; runtime kabulü ayrı.
Bulguların kanıt düzeyi	Üretici kanıtı, yerel ölçüm ve çalıştırılmamış aşama ayrı yazıldı.	Bölüm 20

Yayın kapsamı: Bu rapor birincil bülten ve araştırma yayınlarını teknik olarak sentezler; bağımsız exploit geliştirme, debugger ile gerçek bellek izi veya bütün PHP kaynak yamasının satır satır denetimi yapıldığı iddiası taşımaz. Gerçek CVE sömürüsü bu paketle kanıtlanmış sayılmaz. Bu ayrım raporun doğrulama güvenilirliğinin parçasıdır.

1.3 Önem derecesi ve kurumsal risk ayrımı

Her iki resmî bülten 9,2 ve aşağıdaki CVSS v4.0 vektörünü verir. Ağdan erişim, yetki ve kullanıcı etkileşimi gerekmemesi; yüksek saldırı karmaşıklığı ve ek koşulların varlığıyla birlikte okunur. CVSS puanı bir varlığın iş kritikliğinin veya kurumda ihlal olduğunun ölçümü değildir. [2, 3]

`CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`

2. Zafiyetin Doğası - Neden ve Nasıl Oluşuyor?

2.1 Temel Mekanizma: PHP'nin "Acil Çıkış" Davranışı

Teknik açıklama: PHP, bir HTTP isteği sırasında diziler, dosya yükleme kayıtları, oturum verileri (session data) ve hata bilgileri için bellek ayırır (memory allocation). Bellek yetersizliği (Out of Memory, OOM), yapılandırılmış bellek sınırına (`memory_limit`) ulaşılmasıyla veya alttaki ayırma işleminin başarısızlığıyla oluşabilir. Etkilenen kod yollarında `zend_bailout()` ve `longjmp()`, işlem akışını daha önce belirlenmiş bir kurtarma noktasına taşır. Arada kalan fonksiyonların olağan geri dönüş sırası yürütülmez. [2, 3]

Kesintinin ardından bazı istek kapatma (request shutdown) ve temizleme (cleanup) işlemleri yine yürütülür. Bu işlemler, iç verilerin hâlâ geçerli olduğunu varsaydığında güvenlik sorunu oluşur. Örneğin başlatma yarım kalmışken kapatma işlemi tamamlanmış bir veri yapısıyla karşılaştığını sanabilir. Normal yoldaki oluşturma, kullanma ve temizleme sırası artık güvenilir değildir. Bu nedenle yalnızca HTTP durum koduna bakılarak bellek durumunun güvenli olduğu söylenemez.

Savunma çıkarımı: Hata, uygulamanın bir hata sayfası göstermesiyle düzelmez. Güncelleme PHP'nin bu olağandışı koşullarda nasıl ilerlediğini ve iç durumu ne zaman sıfırladığını değiştirmelidir. Sürüm ve paket doğrulaması, görünür hata metninden daha güçlü kapanış kanıtıdır.

2.2 CVE-2026-109618 - Yarım Kalan Veri Yapısı

Teknik açıklama: Bellek bozulması (Memory Corruption), programın bir bellek alanını beklenen geçerli içerik ve tür dışında işlemesidir. Bu CVE ailesinde iç istek belleği, veri yapısı henüz kısmen başlatılmışken (partially initialized data structure) başka bir işlem tarafından erişilebilir duruma gelebilir. Araya bellek ayırma hatası girdiğinde, kapatma kodu bu yarım durumu tamamlanmış gibi okuyabilir. Veri yapısının hangi alanlarının geçerli olduğuna ilişkin varsayım bozulur. [2]

Kapatma işlemi, geçersiz bir alanı dizi, nesne veya temizlenecek kaynak olarak yorumlarsa yanlış bellek adreslerini izleyebilir. Etki, sürecin çökmesinden programın izleyeceği akışın etkilenmesine kadar uzanabilir. Bellek sınırına ulaşılması tek başına bu sonuçları kanıtlamaz; sonuç, hatanın oluştuğu aşamaya ve çalışma ortamına bağlıdır.

Ağdan gelen istek gövdesi (request body) ile PHP'nin bellekte kullandığı alan da aynı şey değildir. Bir formun ayrıştırılmış gösterimi, gönderilen baytlardan daha fazla yer tutabilir. Bu nedenle tek başına gövde boyutunu küçük tutmak bellek kullanımının aynı ölçüde sınırlı kalacağını kanıtlamaz. [2]

Savunma çıkarımı: OOM günlükleri, işçi yeniden başlamaları ve beklenmeyen hata artışları birlikte izlenmelidir. Bunlar inceleme başlatan sinyallerdir; başarılı sömürünün tek başına kanıtı değildir. Güncelleme, istek boyutu ve kaynak sınırları birbirini tamamlayan kontroller olarak ele alınmalıdır.

2.3 CVE-2026-109619 - Eski Global Durum ve Ömrü Dolan İşaretçiler

Teknik açıklama: İşaretçi (pointer), bellekteki bir adresi tutar. Serbest bırakılmış belleğin kullanılması (Use After Free, UAF), programın artık sahip olmadığı bir bellek alanını eski işaretçi üzerinden kullanmasıdır. PHP'de bir işçi süreci birçok isteği art arda işlerken, her isteğin geçici belleği daha kısa ömürlü olabilir. İşçide kalan global alan, isteğin belleğinden uzun yaşarsa ve zamanında sıfırlanmazsa eski bir adres taşır. Artık geçerli olmayan bu bellek alanını göstermeye devam eden işaretçiye sarkan işaretçi (dangling pointer) denir. [3, 7]

Bu mekanizmada iki rol ayrılır: **üretici (producer)**, daha sonra erişilebilen bir alana istek ömürlü durumu bırakır; **tüketici (consumer)**, sonraki aşamada o eski değeri okur, serbest

bırakır veya ona göre karar verir. Belleğin geri verilmesi ile yeni isteğin normal başlatılması arasındaki sıra önemlidir. Bazı erken istek işlemleri, sıfırlama gerçekleşmeden önce bu alana ulaşabilir.

Serbest bırakılan alan başka bir veri için yeniden ayrılabilir. Eski işaretçiyi kullanan işlem, bu yeni içeriği önceki isteğin geçerli verisi sanarak işleyebilir. Sonuç bazen çökme, bazen yanlış veri erişimi, uygun koşullarda da denetim akışının etkilenmesidir. Her çökme UAF, her UAF da çalışan RCE değildir.

Savunma çıkarımı: Bir isteğin sona ermesi, işçi sürecinin de kapandığı anlamına gelmez. İşçi yeniden kullanımı ve erken başlatma işlemleri bu nedenle denetimde dikkate alınmalıdır. İşçi sayısını artırmak güvenlik yamasının yerine geçmez. [3, 16]

2.4 Neden Bu İki Mekanizma Ayrı CVE?

Teknik açıklama: İlk CVE'nin odağı, OOM kesintisinin iç veriyi geçersiz durumda bırakmasıdır. İkincisinin odağı, belleği artık geçerli olmayan istek durumunun sonraki işleme taşınmasıdır. Birincide başlatma/kapatma tutarlılığı, ikincide durumun ömrü ve sıfırlama zamanı öne çıkar. Her ikisinde bellek güvenliği ihlali görülebilir; UAF terimi yalnızca ikinci CVE'ye özgü bir etiket değildir. [1-3]

Ayrıca eski durum yalnızca OOM ile oluşmayabilir. Ağır hata akışı veya temizleme sırasında yeniden üretilen hata bilgisi de yaşam döngüsünü bozabilir. Bu yüzden iki CVE'yi tek bir form alanına veya yalnızca dosya yüklemeye indirgemek, diğer yolların denetimden kaçmasına neden olur.

Savunma çıkarımı: Güncelleme kaydı iki CVE'yi de kapsamalıdır. Bir belirtiyi durduran ayar, diğer mekanizmanın da kapandığı anlamına gelmez. Denetim, "hangi kontrol hangi yolu azaltıyor?" sorusuna ayrı cevap vermelidir.

2.5 Savunma İçin Neden Önemli?

Teknik açıklama: Yama temel bellek güvenliği hatalarını hedefler. `memory_limit`, `post_max_size`, yükleme ilerleme özelliği (session upload progress) ve serileştirme işleyicileri (serialization handlers) ise belirli koşulları etkiler. Bunların etkisi aynı değildir. Örneğin yükleme ilerlemesini kapatmak belirli erken oturum yollarını azaltabilir; bütün OOM ve eski global durum yollarını kapatmaz. Serileştirme (serialization), veriyi saklanabilir veya aktarılabilir bir biçime dönüştürür; tersine serileştirme (deserialization) bu biçimden tekrar veri yapıları üretir. Oturum serileştirme işleyicisi (session serialization handler), PHP oturum verisinin hangi biçimde kodlanıp çözüleceğini belirler; ilgili ayar `session.serialize_handler` adını taşır. Güvenilmeyen veriyi nesnelere dönüştüren `unserialize()` işlemleri ve yerel dosya dahil etme (Local File Inclusion, LFI) açıkları ayrıca incelenmelidir. [8]

Savunma çıkarımı: Ölçüm, yama, yapılandırma denetimi ve uygulama kontrolünü birlikte yürütün. Kapanış kanıtı olarak paket bülteni, çalışan süreç bilgisi ve tekrarlanabilir kontrol kaydı gerekir.

2.6 Yama hata yaşam döngüsünü nasıl değiştirir

OOM bültenindeki düzeltme, güvenli olmayan noktada kontrol akışını kesen `longjmp()` kullanımını değiştirir. Yumuşak OOM, VM'nin güvenli bir noktasında ele alınır; daha yüksek kaynak eşiğinde sert sonlandırma için `_exit()` kullanılır. Böylece yarım durumun olağan shutdown işlemlerine sunulması önlenir. [2]

Eski global durum için temel hedef, istek belleğinin ömrü ile işçinin sakladığı referansların ömrünü uyumlu tutmaktır. İsteğe ait veri serbest kalırken daha uzun ömürlü durumun bu veriye geçerliymiş gibi erişmemesi gerekir. Hata bilgisini gizlemek, işaretçiyi sıfırlamakla aynı işlem değildir. [3]

Teknik sonuç: Yamalı PHP yine bellek sınırına ulaşabilir. Beklenen değişim belleğin sınırsız olması değil, hata sırasında geçersiz iç durumun kullanılmasının önlenmesidir. Laboratuvardaki OOM gözlemi bu ayrımı öğretir; yamanın bütün güvenlik özelliklerini tek başına ölçmez.

2.7 Sömürülebilirlik koşulları ve etkinin sınırları

Teknik incelemede yalnız “hata var mı?” sorusu yeterli değildir. İlgili istek yolu erişilebilir mi, işlem iç durumu yayımlıyor mu, hata o noktayı kesiyor mu ve geçersiz durum daha sonra tüketiliyor mu soruları ayrı değerlendirilmelidir. Stale-state yolu için aynı işçinin sonraki isteği işlemesi önemli bir koşuldur. Bu açıklama bir bellek yerleştirme veya istismar reçetesi değildir. [2, 3]

İnceleme katmanı	Aranacak bilgi	Yanlış genelleme
Erişilebilirlik	Web SAPI, uygulama yolu ve erken ayrıştırma	Her SAPI’de aynı girdi aynı sonucu verir.
Hata aşaması	Başlangıç, uygulama yürütmesi veya shutdown	Her OOM aynı iç durumu bozar.
Durum ömrü	İstek verisi ile işçi/global durum ömrü	İstek bittiğinde bütün süreç durumu yok olur.
Çalışma düzeni	İşçi yeniden kullanımı, uzantı ve modül bileşimi	Çok işçi kullanımı zafiyeti ortadan kaldırır.
Etki	Süreç çökmesi, veri okuma veya kod çalıştırma için ayrı kanıt	Bir çökme otomatik olarak RCE’dir.

Doğrulanmış RCE’nin yürütme bağlamı web işçisi hesabıdır; bu, kendiliğinden root veya yönetici yetkisi demek değildir. Bununla birlikte web hesabının erişebildiği veritabanı kimlik bilgileri, dosyalar ve ağ bağlantıları sonraki etkiyi büyütebilir. Kod çalıştırma ve bellek açığa çıkması ayrı sonuç türleridir. [2, 4]

2.8 Teknik adlandırma ve yama kapsamı

İki CVE geniş yaşam döngüsü hata sınıflarını kapsar. Wiz’in upload-progress, bellek açığa çıkması, fatal backtrace ve OPcache örnekleri, her bir CVE’nin tek bir fonksiyondan ibaret olduğu şeklinde eşlenmemelidir. Düzeltme kararı iki resmî GHSA bülteni ve sağlayıcının paket kapsamıyla verilir. [2–4]

Kaynak seviyesinde bağımsız inceleme yapılacaksa tam PHP etiketi/commit’i, derleme seçenekleri, uzantılar, hata yolu, çağrı yığını ve yama öncesi/sonrası gözlem kaydedilmelidir. Bu raporda böyle bir incelemenin gerçekleştiği ileri sürülmez. Yalnız sürüm farkından çıkarılan sonuç, kaynak ve bellek düzeyinde regresyon kanıtı değildir.

3. Keşif süreci ve tehdit görünümü

3.1 Araştırma ve koordineli bildirim

Calif'in anlatımı, Ağustos 2026'da bulgu ve PHP'ye bildirim adımlarını; Wiz'in anlatımı ise aynı ay başlayan ayrı araştırma akışını kaydeder. Bulguların bildirilmesi, etkilerinin doğrulanması ve PHP güvenlik ekibiyle koordineli düzeltme süreci bu çalışmaların ortak operasyonel boyutudur. [4, 5]

3.2 Açıklama ve düzeltme zaman çizelgesi

Tarih	Kaynakta bildirilen olay	Kurum için anlamı
Ağustos 2026	Calif ve Wiz'in araştırma ve bildirim süreçleri [4, 5]	Önceden var olan kusur ile kamuya açıklanma tarihi farklıdır.
20 Ekim 2026	PHP yaklaşan kritik sürümü duyurdu; watchTowr erken sızıntıdan söz etti [1, 9]	Kurumun yama hazırlığı teknik ayrıntıları beklememelidir.
21 Ekim 2026	Güvenlik sürümleri yayımlandı [1-3]	Sağlayıcı paketleri ve çalışan servisler doğrulanmalıdır.
21 Ekim 2026	watchTowr honeypot ağında istismar gözlemi bildirdi [9]	Yama ile olay incelemesi birlikte yürütülmelidir.
23 Ekim 2026	PHP ek oturum yapılandırma uyarısını yayımladı [1]	İlk yama sonrasında da üretici duyurusu izlenmelidir.

watchTowr, açıklamanın başlangıçta 22 Ekim için planlandığını, 20 Ekim sızıntısı sonrasında sürümün 21 Ekim'e alındığını aktarır. PHP'nin 20 Ekim duyurusu yayımlama saatini 21 Ekim 14.00 UTC olarak belirtir; Türkiye ve KKTC'de bu tarih için saat 17.00'a karşılık gelir. Bu saat planlanan yayım zamanıdır; her aynanın aynı anda güncellendiği anlamına gelmez. [1, 9]

3.3 Gerçek saldırı bildirimi ile kurumda ihlal kanıtını ayırmak

watchTowr'ın gözlemi 21 Ekim'e uzanırken, AWS 21 Ekim tarihli yayınında MadPot telemetrisini geriye dönük inceleyerek 17 Ekim'e ait bir istismar girişimi belirlediğini bildirir. AWS, aktörün açıklama öncesi bilgiyi nasıl elde ettiğinin bilinmediğini de belirtir. Bu iki yayın gözlem kapsamları farklı olan birincil kaynaklardır. [9, 10]

Bu bulgular tehdidin yalnız teorik olmadığını gösterir; belirli bir kurumun ele geçirildiğini göstermez. Bir tarama, istismar girişimi ve başarılı ihlal ayrı kanıt düzeyleridir. Kurumun kendi günlükleri, süreç kayıtları ve veri erişimleri incelenmeden başarılı sızıntı ilan edilmemelidir.

KEV gibi kataloglar yararlı önceliklendirme kaynaklarıdır; bir kaydın listede bulunmaması güvenlik kanıtı değildir. watchTowr'ın 22 Ekim'e ait KEV ifadesi tarihsel bir durum olarak okunmalıdır; bugünkü katalog durumu olarak tekrarlanmamalıdır.

4. Etkilenen Ortamlar ve Envanter Kontrolü

PHP'nin web sunucusuyla nasıl çalıştığını belirleyen arayüz (Server Application Programming Interface, SAPI), maruziyet koşullarını etkiler. Apache içinde çalışan `mod_php` ile ayrı işçi süreçleri kullanan PHP-FPM (PHP FastCGI Process Manager) denetim kapsamındadır. Diğer PHP web çalıştırıcıları da üretici bülteni ve sağlayıcı paketleri üzerinden kontrol edilmelidir. Resmî PHP duyurusu, uygulama kodundan önce erişilebilen yol nedeniyle mod_php için önceliği özellikle vurgular. [1]

Sürüm ve paket kontrolü

Dal	İlk yamalı kararlı sürüm	Kontrol yaklaşımı
8.2	8.2.35	Bu dalda daha eski sürümleri ve sağlayıcı düzeltmelerini inceleyin.
8.3	8.3.36	Çalışan süreçte sürümü doğrulayın.
8.4	8.4.27	Paket/imaj güncellemesini bütün düğümlerde doğrulayın.

Dal	İlk yamalı kararlı sürüm	Kontrol yaklaşımı
8.5	8.5.12	Yama yanında ek oturum uyarısını değerlendirin.
Daha eski dallar	Desteklenen dala geçiş veya belgeli sağlayıcı düzeltmesi	Resmî güncel sürüm desteği ile kaynak geriye uyarlamasını ayırın.

Tablo, iki CVE için yayımlanan ilk düzeltme eşiklerini gösterir. Kurumlar ilgili dalın güncel güvenlik paketini kullanmalıdır. Eski dallarda desteklenen sürüme geçiş planlanmalı; geçiş beklenirken sağlayıcının güvenlik düzeltmesini eski pakete taşıdığı (backport) durumlar paket bülteniyle belgelenmelidir. Resmî PHP açıklamasında 7.4, 8.0 ve 8.1 için asgari kaynak geriye uyarlamaları da belirtilmiştir. [1-3]

Önerilen yerel envanter komutları:

```
php -v
php --ini
# Debian / Ubuntu
dpkg-query -W 'php*'
# RPM kullanan dağıtımlar
rpm -qa 'php*'
# Konteyner ortamı
docker ps --no-trunc
```

Komut satırı arayüzü (Command-Line Interface, CLI), web servisiyle farklı ikili veya php.ini kullanabilir. Web/FPM ortamında sürüm, SAPI, yüklenen yapılandırma, işçi havuzu (worker pool) ve konteyner imaj kimliği (container image ID) ayrı doğrulanmalıdır. Her varlık için sahip, dış erişim, paket kaynağı, düzeltme kanıtı ve kontrol zamanı tutulmalıdır. Herkese açık phpinfo sayfası bırakmayın.

4.1 Upstream sürümü ile sağlayıcı paketini ayırmak

Aşağıdaki değerler USN-8948-1'in 21 Ekim 2026 tarihli düzeltme örnekleridir. Aynı dağıtımın daha yeni güvenlik paketleri de bulunabilir; tablo sürüm sabitleme talimatı değildir. [11]

İşletim sistemi	İlgili PHP dalı	Bültenin düzeltme paket sürümü
Ubuntu 24.04 LTS	8.3	8.3.6-0ubuntu0.24.04.13
Ubuntu 22.04 LTS	8.1	8.1.2-1ubuntu2.28

Bu örnekte 8.3.6 tabanlı bir paket düzeltme içerebilir. Denetçi yalnız upstream numarasını karşılaştırırsa yamalı sistemi yanlışlıkla açık sayabilir. Tersine, sağlayıcı bülteni olmadan eski bir numaranın otomatik olarak güvenli olduğu da söylenemez.

"PHP 7.3 ve öncesinde hiçbir yama yok" ifadesi de genellenmemelidir. USN-8948-2, belirli eski Ubuntu sürümlerindeki PHP 7.2, 7.0 ve 5.5 paketleri için Pro/Legacy kapsamındaki düzeltmeleri listeler. Bu, bütün eski PHP kurulumlarının desteklendiği anlamına gelmez. Doğru karar işletim sistemi sürümü, depo, tam paket kimliği ve destek kapsamıyla verilir; uzun vadeli hedef desteklenen dala geçiştir. [12]

4.2 SAPI farkları ve kanıtın sınırı

mod_php için OOM kaynaklı, uygulama başlamadan erişilen RCE yolu doğrulanmıştır. FPM başlangıç başarısızlığında çocuğu sonlandırır; bu, aynı başlangıç zincirinin hayatta kalmasını önler. OOM bülteni, o mekanizma için tam FPM exploit'i gösterilmediğini belirtir. Buna karşılık stale-state bülteninde, başarılı başlangıç sonrası uygun uygulama işlemi gerektiren FPM oturum yolu normal dinamik havuzda doğrulanmıştır. [2, 3]

Bu ayrım FPM'nin güvenli olduğu veya yalnız tek işçili havuzların etkilendiği anlamına gelmez. SAPI, uygulama ve hata aşaması birlikte değerlendirilir; farklı yollara ait kanıtlar birbirine taşınmaz.

BÖLÜM II ÇÖZÜM VE KURUMSAL SAVUNMA

5. Kurumların Yapması Gerekenler

Öncelik	İşlem	Sorumlu rol	Tamamlanma kanıtı
Acil	Açık hizmetleri ve PHP çalışma ortamlarını belirleyin	Sistem / uygulama yöneticisi	Varlık, sahip ve sürüm listesi
Acil	PHP güvenlik güncellemesini dağıtın	Sistem / barındırma ekibi	Sağlayıcı bülteni, paket ve çalışan süreç bilgisi
Güncelleme beklerken	Uygun geçici azaltımları uygulayın	Sistem ve güvenlik ekibi	Etkin ayarlar, yeniden başlatma ve işlev kontrolü
Bakım sonrasında	İşlevleri ve tespit kontrollerini sınayın	Uygulama ve güvenlik ekibi	Test kaydı ve alarm örneği
Kapanışta	Kalan işleri ve şüpheli olayları değerlendirin	Hizmet sahibi / güvenlik sorumlusu	Kapanış kaydı veya açık iş planı

Bu sıra, kurumun hizmet kritikliğine ve değişiklik yönetimine göre uygulanmalıdır.

5.1 Acil Yama - Öncelik 1

Neden: Geçici kontroller belirli yolları azaltır; bütün bellek güvenliği hatalarını düzeltmez.

Nasıl: Paket veya konteyner sağlayıcısının tamamlanmış güvenlik sürümünü uygulayın. Önce test ortamında oturum açma, yükleme ve temel iş akışlarını sınayın. Dağıtım sonrasında bütün PHP işçilerini yenileyin; çalışan web süreçlerinin eski sürümle devam etmediğini doğrulayın. Yalnızca diskteki paket sürümüne bakmayın.

Kanıt: Sağlayıcı bülteni, paket/imaj kimliği, çalışan sürüm ve bakım sonrası işlev kontrolünü aynı değişiklik kaydında saklayın. Bütün düğümler doğrulandıktan sonra güncelleme işini kapatın. [1-3]

5.2 Yama Uygulanana Kadar Geçici Azaltma (Mitigation)

Neden: Güncelleme tamamlanana kadar belirli oturum ve hata işleme yollarına erişimi azaltmak.

Nasıl: Gerekmiyorsa `session.upload_progress.enabled=0` ayarını istek ayrıştırmasından önce uygulanan php.ini, havuz veya sunucu yapılandırmasında etkinleştirin; bütün işçileri yeniden başlatın. Uygulama içindeki `ini_set()` veya FPM `.user.ini` dosyasını erken yol için eşdeğer kabul etmeyin. PHP 8.5'te `fatal_error_backtraces=0`, belirli hata geri izi (fatal error backtrace) yolunu azaltır. Web uygulaması güvenlik duvarında (Web Application Firewall, WAF) ilerleme alanını engelleme ek bir katman olabilir. [1, 3]

23 Ekim ek uyarısı: PHP, yapılandırmadaki `php_binary`/`php_serialize` işleyicisinin uygulama sırasında `php` olarak değiştirildiği ve yükleme ilerlemesinin açık kaldığı özel birleşim için, sonraki normal düzeltme sürümüne kadar ilerleme özelliğini kapatmayı öneriyor. Bu ek konu CVE-2026-115560 adıyla anılıyor. Bu ayar birleşimini kullanan kurumlar, yama sonrasında da ilgili üretici uyarısını takip etmelidir. [1]

Sınır: Bu ayarlar bütün varyantlar için kapanış kanıtı değildir. Yapılandırmayı kapatmak, ilgili HTTP isteğinin mutlaka 403 döneceği anlamına da gelmez; özellik devre dışı kalırken normal yükleme çalışmaya devam edebilir.

5.3 Yapılandırma Sertleştirme (Hardening)

Neden: Tek bir sınırı veya hata göstergesini bütün maruziyetin yerine koymamak.

Nasıl: İstek boyutunu, alan ve parça sayısını, iç içe veri derinliğini ve kaynak tüketimini uygulama ihtiyacına göre sınırlandırın. `post_max_size` tek başına yeterli kabul edilmemelidir. Resmî bülten, yama uygulanamıyorsa `memory_limit=-1` ile öngörülebilir PHP sınırı kaynaklı OOM tetikleyicisinin kaldırılmasını geçici seçenek olarak açıklar; alttaki bellek ayırma hataları ve hizmet kullanılabilirliği riski sürer. Bu seçenek kapasite planı ve işletim sistemi/konteyner sınırları değerlendirilmeden topluca uygulanmamalıdır. `RLIMIT_AS` ve `RLIMIT_DATA` eşdeğer önlem değildir. [2]

FPM'de `pm.max_requests=1`, resmî stale-state bülteninde performansın ikincil kaldığı durumlar için anılan, işçi tekrar kullanımını sınırlayan geçici bir seçenektir. Her istekte işçi yenilemenin kapasite ve yanıt süresi maliyeti test edilmelidir; bu ayar güncellemenin yerine geçmez. Serileştirme işleyicilerini tutarlı hale getirmeden önce mevcut oturumlara etkisini sınayın. LFI ve güvenilmeyen unserialize akışlarını ayrıca düzeltin. [3, 16]

5.4 Site sahibi ve barındırma sağlayıcısı için uygulama

WordPress yöneticisi, Site Sağlığı bilgi ekranındaki sunucu bilgisini veya hosting panelinin PHP yönetimini ilk envanter girdisi olarak kullanabilir. Panelde görünen bir seçim, çalışan bütün sitelerin aynı runtime'ı kullandığını kanıtlamaz. Uygulama, yönetim paneli, arka plan işi ve konteynerler ayrı hedefler olabilir.

Paylaşımlı hosting kullanılıyorsa sağlayıcıdan iki CVE için hangi paketlerin güncellendiğini, eski işçilerin yenilendiğini ve web isteklerinin yamalı ortamda çalıştığını yazılı olarak isteyin. Yanıtı kurumun varlık kaydına ekleyin. Kendi sunucusunu yöneten ekip ise her PHP dalını eşleşen modülleriyle birlikte güncellemelidir.

CyberPanel örneği: Üretici, PHP çekirdeği güncellenip eski OPcache modülünün bırakılmasının eksik dağıtım olduğunu belirtir. Ayrıca kendi test ortamında panel/phpMyAdmin için kullanılan `/usr/local/lscp/fcgi-bin/lsphp` kopyasını ayrı yenilediğini bildirir. Bu yol bütün sunucular için körlemesine dosya kopyalama talimatı değildir; doğru runtime ve bağımlılıklar sağlayıcı yönergesiyle belirlenir. [13]

5.5 Ayarlar için neden etki ve sınır matrisi

Kontrol	Beklenen katkı	Kabul edilmeyecek çıkarım
Yamalı PHP ve eşleşen modüller	Temel güvenlik düzeltmesinin dağıtılması	Diskteki paket güncellendi diye eski işçinin yenilendiği
Erken session.upload_progress.enabled=0	Doğrulanmış oturum yollarını azaltma	Dosya yüklemenin tamamen kapandığı
PHP 8.5 fatal_error_backtraces=0	Belirli backtrace yolunu azaltma	Bütün global durum hatalarının giderildiği
Yönetimden memory_limit=-1	Kontrollü PHP sınırıyla OOM oluşumunu azaltma	Alttaki ayırma hatalarının veya DoS riskinin yok olduğu
Ön uçta istek ve parça sınırları	Kaynak tüketimini ve belirli yolları sınırlama	İstek baytı ile bellek tüketiminin aynı olduğu
WAF ve IDS	Görülen trafikte eşleşme ve inceleme sinyali	Bütün erişim yollarının korunduğu

Resmî PHP duyurusu, yükleme gerekmeyen uygulamalar için `post_max_size=8M`, gerekli olanlar için `32M` örneklerini verir; `upload_max_filesize` bunun altında kalmalıdır. Bunlar uygulama

ihtiyacıyla test edilerek seçilmelidir. Küçük bir HTTP gövdesinin PHP'de aynı miktarda bellek kullanacağı varsayılmamalıdır. [1, 2]

OOM için `memory_limit=-1` yalnız kontrollü PHP sınırıyla ilgili tetikleyiciyi kaldırır; alttaki bellek ayırma hatasını kesin önlemez. Kullanılabilirlik sınırını ayrıca tasarlayın. Hata gösterimini kapatmak, `error_reporting` seviyesini düşürmek veya `@` kullanmak eski durumun tüketilmesini engellemez. [2, 3]

5.6 CDN arkasındaki gerçek sunucuyu kapsamak

CDN veya WAF üzerinden görülen alan adı kadar, asıl sunucuya doğrudan erişilen yollar da envantere alınmalıdır. Kurumun izin verdiği yönetim, sağlık kontrolü ve servis bağlantıları korunarak origin erişimi tasarlanmalıdır. Bu raporun denetim önerisi, bütün web yollarının aynı koruma katmanından geçtiğinin varsayılmamasıdır.

Örnek: Ön yüzde WAF alarmı görülürken doğrudan origin IP'sine gelen istek o WAF'a uğramıyorsa, alarmın varlığı ikinci yol için koruma kanıtı olmaz. Aynı mantık alt alan adları, alternatif portlar ve ayrı yönetim panelleri için de geçerlidir.

5.7 Değişiklik planı ve teknik kabul kapısı

Yama dağıtımında işletim sistemi PHP'si, özel dizine kurulu LSAPI, panel/phpMyAdmin PHP'si, arka plan süreçleri ve konteyner imajları ayrı varlıklar olarak izlenir. OPcache dahil etkilenen modüller aynı güvenlik sürümüyle uyumlu olmalıdır. [13]

Kurumun kabul kapısı şu kanıtları birlikte gerektirir: sağlayıcının iki CVE'yi kapsayan kaydı; kurulu paket/imaj kimliği; eski işçilerin yenilendiği; gerçek web yolunun doğru runtime ve modülleri kullandığı; oturum, yükleme ve kritik iş akışının çalıştığı; kalan şüpheli olayların ayrı takip edildiği.

Geri alma planı uygulama uyumluluğu için gereklidir. Ancak açık sürüme dönmek güvenlik açısından düzeltme sayılmaz. Böyle bir durumda hizmetin erişimi sınırlandırılır veya geçici durdurma kararı hizmet sahibiyle verilir; geri dönüşün gerekçesi ve kalan risk kaydedilir. Bütün düğümler doğrulanmadan merkezi iş kapatılmaz.

5.8 Kamu ve özel sektör için önceliklendirme

Durum	Operasyonel yaklaşım
İnternete açık ve düzeltmesi bilinmeyen hizmet	Kritik müdahale sırasına alın; sahip ve runtime kanıtını hemen çıkarın.
Kimlik, sağlık, finans veya kurum içi sır işleyen hizmet	Yama çalışmasına olay incelemesi ve erişim kanıtı değerlendirmesini ekleyin.
Paylaşımlı barındırma veya çok kiracılı yapı	Sağlayıcı sorumluluğunu ve tenant sınırlarını açıkça doğrulayın.
İç ağ hizmeti	"İçeride" diye kapsamdan çıkarmayın; erişim ve bağımlılıkları inceleyin.
Destek dışı veya iş kritikliğinden güncellenemeyen sistem	Sahibi, istisna süresi ve erişim azaltımı olan geçiş planı oluşturun.

Bu tablo SnipeLab'ın operasyonel önerisidir; belirli bir sektör için mevzuat uygunluk belgesi değildir. Kurumun olay bildirim ve veri koruma yükümlülükleri kendi yetkili birimlerince değerlendirilir.

6. Tespit ve Olay İncelemesi

İzlenecek belirtiler

Bir duman dedektörü tehlikeyi haber verebilir; arızalı tesisatı onarmak için ayrıca müdahale gerekir. Tespit kontrolleri de yama çalışmasını destekler ve şüpheli olayların incelenmesine yardımcı olur.

Saldırı tespit sistemi (Intrusion Detection System, IDS), WAF ve güvenlik bilgi ve olay yönetimi (Security Information and Event Management, SIEM) kayıtlarını birlikte değerlendirin.

Sinyal	Birlikte incelenecek kanıt	Yapılacak işlem
OOM / bellek sınırı hatası	İstek zamanı, bellek kullanımı, işçi kimliği	Olağan uygulama hatası ile şüpheli istek ilişkisini araştırın.
HTTP 500/502 artışı	Ters vekil (reverse proxy), PHP/FPM ve yeniden başlama kayıtları	Etkilenen uç noktaları ve zaman aralığını belirleyin.
Olağandışı çok parçalı form (multipart/form-data) trafiği	İstek sıklığı, boyutu ve normal yükleme profili	İlgili WAF/IDS olaylarıyla eşleştirin.
Anormal oturum çerezleri (session cookies) veya çözümleme hataları	Oturum, uygulama ve işçi kayıtları	Aynı zaman aralığındaki istekleri inceleyin.
PHP'nin beklenmeyen alt süreç oluşturması	Süreç ağacı (process tree), komut satırı ve uygulamanın normal işleri	Yetkisiz kod çalıştırma olasılığı için olay incelemesi başlatın.

Bu sinyaller uygulama hatalarında veya meşru iş akışlarında da oluşabilir. İncelemede istek, süreç ve günlük kanıtlarını birlikte kullanın.

Tespit kontrollerini devreye alma

Suricata/Snort, Sigma, ModSecurity ve OpenResty için topluluk materyalleri bulunmaktadır. İlgili kuralların ürün uyumluluğu ve kapsamı bağımsız olarak doğrulanmadı. Kullanılacak dosyayı ürün sürümüne ve uygulamanın normal davranışına göre inceleyin. [6]

1. Kural dosyasının kaynağını ve sürümünü kaydedin; mevcut yapılandırmayı yedekleyin.
2. Ürünün sözdizimi kontrolünü (syntax check) çalıştırın. Kural kimliği çakışmalarını ve SIEM alan eşlemelerini düzeltin.
3. Test ortamında zararsız bir eşleşme girdisi ve normal uygulama isteği kullanın.
4. Alarmın doğru kural kimliği, hedef ve zamanla kaydedildiğini doğrulayın.
5. Uygun kuralları engelleme moduna alın; yükleme ve oturum işlevlerini yeniden kontrol edin.

Görünürlük kontrolü: Erişim günlüklerinin istek gövdesini kaydettiğini varsaymayın. Şifreli trafik, gövde inceleme sınırı ve güvenlik cihazının ayrıştırma davranışı tespiti etkiler. Seçilen kontrolün gerçekten gördüğü trafik üzerinde çalıştığını doğrulayın.

6.1 Olay incelemesinde kanıtı koruma sırası

Şüpheli istismar varsa yama ve olay müdahalesi iki ayrı iş akışı olarak izlenir. Önce olay zamanı, hedef varlık, istek yolu, işçi kimliği, süreç ağacı ve ilgili günlük kaynakları korunur. Aktif zarar varsa erişim sınırlama öncelik kazanabilir. Yeniden başlatma uçucu kanıtı yok edebileceğinden, mümkün olan ortamda yetkili olay ekibi kanıt toplama ile müdahale sırasını belirler.

Bellek dökümleri ve ayrıntılı istek kayıtları kişisel veri veya sır içerebilir. Erişimi sınırlandırın; toplama gerekçesi, bütünlük özeti, zaman ve teslim zinciri kaydedilsin. Toplu ve gereksiz veri kaydını varsayılan yapmayın.

Doğrulanmış ihlal veya güvenilir bellek erişimi bulgusu varsa, erişilmiş olabilecek oturum ve servis sırlarının iptal/yenileme kapsamını belirleyin. Her alarmda bütün kurum sırlarını değiştirme veya her yamadan sonra “olay kapandı” deme yaklaşımı yerine kanıta dayalı kapsamlandırma kullanın. Kurtarma ve yeniden yayına alma, temiz runtime ile uygulama bütünlüğünü de kapsamalıdır.

6.2 Algılama kapsamı ve yanlış negatifler

Olay kaydına görünürlük alanını yazın: trafik hangi TLS sonlandırıcısından ve WAF’tan geçti, gövde incelenebildi mi, hangi limitte kesildi, origin’e alternatif yol var mı? Test edilmeyen yol “alarm yok” diye güvenli sınıflandırılmaz. Meşru uygulamanın upload-progress kullanması yanlış pozitif üretebilir; işaretçisiz normal yükleme karşı kontrolü bu nedenle zorunludur.

7. Denetim Kontrol Listesi

- ☐ Bütün PHP varlıkları, sahipleri ve dış erişimleri kaydedildi.
- ☐ CLI ve gerçek web/FPM ortamı ayrı doğrulandı.
- ☐ Sağlayıcı paketinin iki CVE düzeltmesi belgelendi.
- ☐ Güncelleme bütün düğümlere dağıtıldı; eski işçiler yenilendi.
- ☐ Oturum açma, yükleme ve temel iş akışları yeniden sınandı.
- ☐ Erken yükleme ilerleme ayarı ve ek oturum uyarısı değerlendirildi.
- ☐ LFI, unserialize ve işleyici değişiklikleri incelendi.
- ☐ IDS/WAF/SIEM sözdizimi, alan eşleme ve alarm kontrolleri tamamlandı.
- ☐ OOM, HTTP hata ve süreç bulguları birlikte incelendi.
- ☐ Kapanış kanıtı, kalan işler, sorumlular ve tarih kaydedildi.

Bu liste önerilen kurum denetim akışıdır. “WAF açık”, “200 döndü” veya “hata görülmedi” tek başına kapanış ölçütü değildir.

8. Kurumsal İletişim ve Farkındalık

Teknik ekip: Hangi varlıkta hangi sürüm/SAPI bulunduğu, hangi ayarın hangi yolu azalttığı ve yamanın nasıl doğrulandığı paylaşılmalıdır. Bir çökme, bir alarm ve doğrulanmış kod çalıştırma ayrı sonuç türleridir.

Yönetim: Toplam varlık sayısı, düzeltmesi doğrulanan ve bekleyen hizmetler, bakım etkisi, sorumlular ve kalan risk raporlanmalıdır. Kuruma ait kayıtlar incelenmeden veri sızıntısı veya aktif ihlal ilan edilmemelidir.

Kullanıcılar: Doğrulanmış hizmet etkisi, planlanan bakım ve gerekiyorsa kullanıcı işlemi açıklanmalıdır. Teknik şüphe, kesin olay gibi sunulmamalıdır.

BÖLÜM III UYGULAMALI SNİPELAB LABORATUVARI

9. SnipeLab doğrulama kapsamı ve kanıt sınırı

Bu doğrulama çalışması, aynı küçük uygulamanın eski PHP, yamalı PHP ve azaltım uygulanmış eski PHP üzerinde çalıştırılmasını sağlar. Operatör gerçek HTTP çalışma ortamını inceler, CLI farkını kontrol eder, ayarı doğrular ve normal yüklemenin sürmesini sınar. İsteğe bağlı sabit boyutlu OOM deneyi, günlük toplama ve hata sonrası hizmet kontrolünü kapsar.

Bu uygulamalı bölüm, yama ve yapılandırma doğrulama laboratuvarıdır. Özel bellek düzeni oluşturmaz, eski işaretçi tüketmez, komut çalıştırmaz ve süreç belleği sızdırmaz. Eski sürümü göstermek maruziyet envanteridir; kontrolün alarm vermesi tespit doğrulamasıdır; bellek sınırına ulaşmak OOM gözlemidir. Bunların hiçbiri tek başına CVE sömürüsünü doğrulamaz.

Deney	Ne gösterir?	Ne göstermez?
Sürüm ve SAPI	Gerçek HTTP çalıştırıcısının kimliği	Sömürü başarısı
Azaltım ayarı	Erken yapılandırmanın devrede olması	İki CVE'nin tamamen kapandığı
Normal yükleme	Değişiklik sonrası işlevin sürmesi	Bütün yükleme senaryolarının güvenliği
Kontrollü OOM	Kaynak sınırı ve günlük akışı	UAF, bellek bozulması veya RCE
WAF işaretçisi	Seçilen imza ve günlük zinciri	Bütün yolların engellendiği

Deney kaydı: Beklenen sonuç ile gerçek ölçüm ayrılır. Operatör sonuç dosyasını, günlükleri, imaj kimliklerini ve etkin yapılandırmayı aynı kayıt altında saklar. Kodlar bu teslimde Ek A ve paket içinde sağlanmıştır.

9.1 Yaşam döngüsü modelinin uygulamalı gösterimi

`tools/lifecycle_model.py`, önceki isteğe ait bir referansın veri silindikten sonra kaldığı durumu görünür kılar. İlk senaryoda invariant bozulur; ikinci senaryoda referans veri serbest kalmadan önce sıfırlanır. Çıktı iki senaryoyu aynı JSON içinde verir.

```
python3 tools/lifecycle_model.py > evidence/lifecycle-model.json
```

Bu araç Python nesnelerindeki kimlik ilişkisini modeller. PHP motoru, gerçek serbest bellek, RCE veya CVE tetikleyicisi çalıştırmaz. İşlevi, teknik mekanizmayı ekip içinde aynı kavramlarla tartışılabilir hale getirmektir.

10. Ortamı hazırlayın

Önerilen izole doğrulama ortamı: ayrı Ubuntu sanal makinesi, örneğin 4 sanal işlemci, 8 GB RAM, 20 GB boş disk. Bunlar bu rehberin planlama değerleridir. GPU gerekmez. Docker Engine ve Compose eklentisi kurulu olmalıdır; sürümlerini kaydedin:

```
docker version
docker compose version
python3 --version
```

Teslim edilen SnipeLab paketini açın; `php2shell-lab` içine girin. Dosya listesini Ek A ile karşılaştırın. İlk kabul adımında `SHA256SUMS` doğrulanır:

```
cd php2shell-lab
sha256sum -c SHA256SUMS
mkdir -p evidence
```

Sanal makinede gerçek kurum verisi veya üretim kimlik bilgisi bulundurmayın. Dosyalar yalnızca 127.0.0.1 adresinde port yayımlar; Compose ağı `internal` olarak tanımlıdır. Bu, bir sanal makine güvenlik sınırının yerine geçmez. Docker'ın olağan bridge ağı dış erişime izin verebildiğinden, `internal` ayarını kendi çıktınızda doğrulayın. Kaynak: [15].

11. İmajları alın ve kimliklerini kaydedin

```
docker compose config > evidence/compose-resolved.yaml
docker compose pull apache-old apache-fixed apache-mitigated
docker image inspect php:8.5.11-apache php:8.5.12-apache > evidence/images-apache.json
python3 tools/collect.py --lock
docker compose --env-file images.lock.env config > evidence/compose-locked.yaml
```

Kullanılan PHP sürümleri kaynaklarda görülen karşılaştırma çiftidir. Etiketlerin bu ortamda çekilebilirliği denenmemiştir. Çekme başarısız olursa başka sürümü sessizce yerine koymayın. Hangi etiketin bulunamadığını kaydedin; sürüm değişirse beklenti tablosunu ve denetleyiciyi birlikte güncelleyin. `latest` kullanmayın. Nginx için `1.28.0-alpine` etiketi seçilmiştir; bunun da gerçek RepoDigest değeri kaydedilip kilitlenir. Bu Nginx sürümü üretim tavsiyesi değil, laboratuvar girdisidir.

İmajlar alındıktan sonra sanal makinenin dış ağ erişimini kapatabilirsiniz; hedef servisler için ayrı `internal` ağ kullanılır. Uygulama klasörü konteynerlere salt okunur bağlanır. Docker soketi bağlanmaz; ayrıcalıklı konteyner kullanılmaz.

Resmî PHP imajı bilgileri: [14].

12. Üç Apache ortamını başlatın

```
docker compose --env-file images.lock.env up -d apache-old apache-fixed apache-mitigated
docker compose ps > evidence/containers.txt
docker network inspect php2shell-defense-lab_lab > evidence/network.json
```

`network.json` içinde `Internal: true` görülmelidir. `docker compose ps` yayımlanan portların 127.0.0.1'e bağlı olduğunu göstermelidir. Başlatma hatasında:

```
docker compose logs --no-color > evidence/startup.log
```

Ortam	Adres	Beklenen PHP	İlerleme özelliği
apache-old	http://127.0.0.1:18081/status.php	8.5.11	Açık
apache-fixed	http://127.0.0.1:18082/status.php	8.5.12	Açık
apache-mitigated	http://127.0.0.1:18083/status.php	8.5.11	Kapalı

Yamalı ortamda özelliğin açık olması yalnızca deney karşılaştırmasını temiz tutmak içindir. Üretim önerisi olarak okunmamalıdır; Bölüm 5.2'deki 23 Ekim ek uyarısını ayrıca uygulayın.

13. Gerçek web ortamını kontrol edin

```
curl --fail --silent --show-error http://127.0.0.1:18081/status.php
curl --fail --silent --show-error http://127.0.0.1:18082/status.php
curl --fail --silent --show-error http://127.0.0.1:18083/status.php
```

Her cevapta `php_version`, `sapi`, `ini_file`, `ini_scanned`, `upload_progress`, `fatal_backtraces` ve bellek sınırını inceleyin. Apache SAPI değeri `apache2handler` olmalıdır. HTTP çıktısının `PHP2Shell` açıklamalarında kullanılan çalışma ortamıyla eşleştiğini kontrol edin. Azaltım ortamında sonradan yüklenen `99-mitigation.ini` listelenmeli; ilerleme ve geri izleme değerleri kapalı olmalıdır.

CLI karşılaştırması:

```
docker compose --env-file images.lock.env exec -T apache-fixed php -v
docker compose --env-file images.lock.env exec -T apache-fixed php --ini
```

CLI çıktısının web SAPI'siyle aynı olmadığına dikkat edin. `status.php` bilinçli olarak küçük bilgi verir; doğrulama bitince kaldırılmalıdır.

14. Otomatik savunma kontrollerini çalıştırın

```
python3 tools/check.py
```

Denetleyici yalnızca kod içindeki sabit localhost hedeflerini kullanır ve ortam proxy'sini atlar. Dış hedef argümanı yoktur. Sürüm, SAPI, özellik ayarı ve küçük normal yükleme kontrol edilir. `security_posture` alanı eski, kısmen azaltımlı ve iki CVE için yamalı karşılaştırma hedefini ayırır. Sonuç `evidence/checks-<UTC-zamanı>.json` dosyasına yazılır. Başarısız kontrol sıfırdan farklı çıkış kodu verir.

Beklenen: Üç ortam da kendi beklentisine göre PASS verir. Buradaki PASS, eski ortamın güvenli olduğu anlamına gelmez; 8.5.11 beklenen kontrol hedefinde gerçekten 8.5.11 bulunduğu anlamına gelir. Sonuç dosyasındaki `scope` alanı bu ayrımı belirtir.

Azaltım ortamında normal yüklemenin çalışması beklenir. `session.upload_progress.enabled=0`, dosya yüklemeyi kapatmaz ve alan içeren isteği otomatik olarak reddetmez. İlerleme özelliğinin kapandığı, etkin ayar üzerinden doğrulanır.

14.1 Sonuç alanları ve başarısız kontrolün anlamı

`check.py` Python standart kütüphanesiyle çalışır; dış bağımlılık istemez. Hedef listesi sabit loopback portlarından oluşur, ortam proxy'si kullanılmaz, HTTP yönlendirmeleri izlenmez ve yanıt boyutu sınırlıdır. Normal yüklemede dosya özeti, boyut ve kalıcı kaydedilmeme alanı birlikte kontrol edilir.

`pass`, yalnız seçilmiş karşılaştırma beklentilerinin sonucudur. `security_posture` eski hedef için `known_affected_comparison`, azaltımlı eski hedef için `known_affected_partial_mitigation`, yamalı hedef için `patched_for_two_cves_only` olur. Bu son değer 23 Ekim ek konusunu veya gelecekteki açıkları otomatik kapatmaz.

OOM isteğinde `oom_confirmed=false` bırakılır; otomatik aracı günlük incelemesi yapılmış gibi göstermekten kaçınır. Operatör günlükte işaretçi, fatal hata/işçi davranışı ve sonraki sağlık kontrolünü ilişkilendirir. Ağ hatası veya HTTP 500 tek başına OOM doğrulaması değildir.

15. İsteğe bağlı: sınırlı OOM ve günlük gözlemi

Bu bölüm yalnızca ayrılmış laboratuvar sanal makinesinde yürütülür. Deney uygulama kodunda `memory_limit` değerini 16M yapar ve en fazla 24 MiB sabit tahsis ister. Ağ isteğinden tahsis büyüklüğü alınmaz. Özel multipart, sahte işaretçi veya bellek yeniden kullanım zinciri yoktur.

```
python3 tools/check.py --oom-demo
docker compose logs --no-color > evidence/oom-observation.log
docker stats --no-stream > evidence/resources.txt
```

Günlükte `LAB_BOUNDED_OOM_DEMO` işaretini, ardından bellek sınırı hatası veya işçi çıkışını arayın. Önceki ve sonraki `status.php` çağrılarının PID alanlarını karşılaştırın. İşçi çıkmışsa sonraki isteğin yeni işçiye ulaşması normal olabilir. Çıkış biçimi ve HTTP kodu derleme ile hata yoluna bağlıdır; belirli bir kodu evrensel başarı şartı saymayın.

Yorum: Yamalı sürümde de bellek sınırı hatası olabilir. Yama, sınırsız bellek sağlamaz; güvenli olmayan hata yaşam döngüsünü düzeltir. "İki ortam da OOM verdi, yama işe yaramıyor" sonucu yanlıştır. Bu deney CVE'nin olumsuz veya olumlu sömürü kontrolü değildir.

16. İsteğe bağlı: PHP-FPM karşılaştırması

```
docker compose --profile fpm pull
docker image inspect php:8.5.11-fpm php:8.5.12-fpm nginx:1.28.0-alpine > evidence/images-fpm.json
```

```
python3 tools/collect.py --fpm --lock
docker compose --env-file images.lock.env --profile fpm up -d
docker compose --env-file images.lock.env exec -T gateway nginx -t
python3 tools/check.py --fpm
```

Eski FPM http://127.0.0.1:18084/status.php, yamalı FPM http://127.0.0.1:18085/status.php adresindedir. Her ikisinde `sapi=fpm-fcgi` beklenir. FPM'nin 9000 portu ana makineye yayımlanmaz; Nginx yalnızca iç ağdan erişir.

Varsayılan havuz değiştirilmez. Tek işçili havuz kurmak bu rehberin savunma kontrolü değildir. Arka arkaya görülen PID değerleri yalnızca işçi yönlendirmesini gözlemletir; eski global duruma erişildiğini kanıtlamaz. FPM ve mod_php sonuçlarını tek bir hata koduyla genellemeyin.

17. Tespit sistemini ayrı doğrulayın

Bu pakette Suricata, ModSecurity veya SIEM kurulu değildir. Aşağıdaki adımlar mevcut test sensörünüz içindir; sensör olmadan bu bölümü "geçti" işaretlemeyin.

1. ikow deposunun `detection` dosyalarını alın; commit ve dosya hash'lerini kaydedin. Kaynak: [6].
2. Kendi kopyanızdaki kural sayısını, kimliklerini, lisansı ve commit bilgisini kaydedin. Kural sayısını yalnız kullandığınız commit üzerinden raporlayın.
3. Ürüne uygun sözdizimi kontrolü yapın. Suricata/Snort ortak dosyasını bütün sürümlerde doğrudan uyumlu varsaymayın. Sigma kimliklerini kullandığınız araç zincirinin gereksinimleriyle karşılaştırın.
4. Kuralları önce testte alarm moduna alın. Trafiğin sensörden geçtiğini ve HTTP gövdesinin görülebildiğini doğrulayın. Ana makine loopback trafiği fiziksel sensöre kendiliğinden gitmez.
5. Aşağıdaki küçük işaretçi isteğini, yalnızca kendi localhost test WAF'ınızın yönlendirdiği adrese gönderin. Adres örneğini üretim hedefiyle değiştirmeyin.

```
# 18081 dogrudan Apache'dir; WAF ancak siz bu yolun onune yerlestirirseniz alarm uretebilir.
curl --silent --show-error \
  -F 'PHP_SESSION_UPLOAD_PROGRESS=lab-marker' \
  http://127.0.0.1:18081/status.php
```

Bu istek yükleme ilerlemesi adını taşıyan zararsız küçük bir işaretçidir. Bir imzanın eşleşmesi beklenebilir; CVE tetiklendiği iddia edilmez. Alarm modu 200 cevabı verebilir; engelleme modu ürün politikasına göre farklı cevap döndürebilir. Kesin sonuç, kural kimliği ve zaman damgasıyla olay günlüğünden alınır. İstek başarılı olup alarm görünmüyorsa önce trafik yolu, kuralın yüklenmesi ve gövde görünürlüğünü inceleyin.

6. İşaretçiyi içermeyen normal yüklemeyi `python3 tools/check.py` ile tekrarlayın. Meşru uygulamanın bu alanı kullanıp kullanmadığını ayrıca kontrol edin; sıfır yanlış pozitif varsaymayın.
7. Eşleşme ve normal kontrol için zaman, kural kimliği, karar, HTTP cevabı ve günlük kaynağını rapora yazın. Tek testle bütün WAF atlatmalarının kapandığını ilan etmeyin.

18. Operatör doğrulama ve kabul kartı

Kontrol	Beklenti	Gerçek sonuç / kanıt
Eski Apache	8.5.11, apache2handler	Doldurulacak
Yamalı Apache	8.5.12, apache2handler	Doldurulacak
Azaltım	Eski sürüm; ilerleme ve fatal geri izi kapalı	Doldurulacak
Normal yükleme	Küçük dosya kabul edilir, kalıcı kaydedilmez	Doldurulacak

Kontrol	Beklenti	Gerçek sonuç / kanıt
OOM gözlemi	İşaretli günlük; sonrasında hizmet kontrolü	Yapıldı / yapılmadı
FPM	fpm-fcgi; doğru eski/yamalı sürümler	Yapıldı / yapılmadı
Tespit sistemi	Gerçek alarm kaydı ve normal kontrol	Yapıldı / yapılmadı
UAF/RCE	Bu rehberde sınanmadı	Kanıtlanmadı

Sunumda kullanılabilecek doğru sonuç: “Eski ve yamalı çalışma ortamlarını ayrı doğruladık; geçici ayarın devrede olduğunu ve normal yüklemenin çalıştığını gösterdik. Kontrollü bellek hatasını ve günlük akışını gözlemledik. Bu testler kod çalıştırma kanıtı değildir.” Son cümleleri yalnızca gerçekten yürütülen adımlara göre düzenleyin.

19. Kapatma ve temizleme

Önce kanıtları saklayın:

```
docker compose --env-file images.lock.env --profile fpm logs --no-color > evidence/final.log
docker compose --env-file images.lock.env --profile fpm ps -a > evidence/final-containers.txt
docker compose --env-file images.lock.env --profile fpm down
```

Bu komutlar yalnızca bu Compose projesini kapatır; genel Docker temizleme komutu verilmemiştir. Laboratuvar tanılama dosyalarını üretime taşımayın. Sonraki denemede aynı imaj kimliklerini kullanın ve yeni bir sonuç dosyası üretin.

BÖLÜM IV KABUL KANITLARI VE TEKNİK REFERANSLAR

20. Doğrulama durumu ve teknik sınırlılıklar

Aşama	Bu sürümde gerçek durum	Tamamlanma ölçütü
Python derleme ve HTTP kontrol aracı	8/8 sözleşme testi geçti.	Paket tests/test_controls.py
Olumlu ve olumsuz HTTP senaryoları	Beş gerçek localhost fixture'ı ile yürütüldü.	Yanlış sürüm, bozuk hash ve yönlendirme reddi
Compose YAML	Ayrıştırma ve yapısal ağ/port/bağlantı/kaynak kontrolü geçti.	Docker Compose runtime doğrulaması ayrıca gerekir.
Soyut yaşam döngüsü modeli	İki invariant senaryosu geçti.	Gerçek PHP UAF testi değildir.
Docker Compose config, imaj ve digest	Bu kontrol testlerinde çalıştırılmadı.	Docker ortamında operatör kabulü gerekir.
PHP sözdizimi ve Apache/FPM HTTP	Bu kontrol testlerinde çalıştırılmadı.	Gerçek PHP runtime kabulü gerekir.
Nginx sözdizimi ve gerçek OOM	Çalıştırılmadı.	nginx -t ve runtime günlük korelasyonu
WAF/IDS/SIEM	Gerçek sensör alarmı çalıştırılmadı.	Alarm ve normal kontrol kaydı gerekir.
Gerçek PHP UAF/RCE veya bellek sızıntısı	Bu paketin test kapsamına dahil değil.	Bu raporda bağımsız kanıt ileri sürülmez.

20.1 Operatör kabul sırası

1. Paket SHA256 listesi, dosya bütünlüğü ve Python sözleşme testleri.
2. `docker compose config`, imaj çekme ve gerçek digest kilidi.
3. Apache servislerinin başlatılması, log ve iç ağ kontrolü.
4. Konteynerde PHP sözdizimi kontrolü, sonra normal HTTP kontrolleri.
5. İsteğe bağlı FPM ve Nginx kontrolü.
6. İsteğe bağlı OOM ve tespit kontrolleri; kanıtların ayrı sınıflandırılması.
7. Günlük ve envanterin saklanması; yalnız proje servislerinin kapatılması.

Paketin test araçları servislerle aynı portları kullanır; fixture testlerini gerçek laboratuvar servisleri kapalıyken çalıştırın.

```
python3 -m unittest discover -s tests -v
docker compose --env-file images.lock.env exec -T apache-old php -l /var/www/html/status.php
docker compose --env-file images.lock.env exec -T apache-old php -l /var/www/html/upload.php
docker compose --env-file images.lock.env exec -T apache-old php -l /var/www/html/oom.php
```

Aynı sözdizimi kontrollerini diğer gerçek PHP hedeflerinde de tamamlayın. Dosyanın bir dalda lint edilmesi diğer dalların çalışma kanıtı değildir. Runtime kabulü tamamlanana kadar "SnipeLab'da gerçek PHP deneyleri başarıyla yürütüldü" ifadesi kullanılmaz.

21. Kurumsal kapanış kaydı örneği

Bu şema kurumun değişiklik kaydına uyarlanabilir. Alanların boş kalması veya yalnız "güvenli" yazılması doğrulama sayılmaz.

Kayıt alanı	Doldurulacak bilgi
Varlık ve hizmet sahibi	Sunucu, uygulama, düğüm ve sorumlu ekip
Önceki durum	Paket kaynağı, sürüm, SAPI ve dış erişim
Uygulanan işlem	Sağlayıcı bülteni, yeni paket/imaj, ilgili modüller
Çalışan ortam kanıtı	İşçi yenileme zamanı ve HTTP doğrulaması
İşlev kontrolü	Oturum, yükleme ve temel iş akışının sonucu
Tespit ve inceleme	Alarm kontrolü, zaman aralığı ve şüpheli bulgular
Kalan işler	Sorumlu, hedef tarih ve iş takip kaydı
Kapanış	Onaylayan hizmet sahibi, tarih ve kanıt konumu

22. Terimler ve rapor okuma notları

Terim	Bu rapordaki anlamı
OOM	Bellek sınırı veya ayırma başarısızlığıyla ilişkili bellek yetersizliği durumu
UAF	Serbest bırakılmış belleğe eski referans üzerinden erişim
RCE	Uzak girdinin hedef süreçte kod çalıştırmaya yol açması
SAPI	PHP’nin web sunucusu veya diğer yürütme ortamlarıyla çalışma arayüzü
Worker	Bir veya daha fazla isteği işleyen PHP süreci
Backport	Güvenlik düzeltmesinin eski sürüm temeline sağlayıcı tarafından taşınması
Origin	CDN veya vekil arkasındaki asıl uygulama sunucusu
İmaj digest	Konteyner imajını içeriğiyle tanımlayan özet kimliği
PASS	Belirli bir kontrolün beklentiye uyduğunu gösteren sonuç; genel güvenlik belgesi değildir.

Başarılı sömürü, veri sızıntısı ve kurumsal kapanış kararı ilgili teknik kanıta dayanmalıdır.

23. Kaynaklar

Erişim ve teknik bilgi kesim tarihi 25 Ekim 2026’dır. Tam URL’ler basılı nüsha için korunmuştur. Birincil duyurular, sağlayıcı açıklamaları ve içerik kontrolü yapılamayan topluluk materyalleri aşağıda ayrılmıştır.

[1] PHP Project 2026 güvenlik ve ek oturum duyuruları. 21 Ekim yama eşikleri ve 23 Ekim ek uyarısı.
https://www.php.net/archive/2026.php

[2] PHP GHSA w8hw w9fx j79w. CVE-2026-109618 için resmî OOM bülteni.
https://github.com/php/php-src/security/advisories/GHSA-w8hw-w9fx-j79w

[3] PHP GHSA 6wmf 4rv5 p3rp. CVE-2026-109619 için resmî stale-state bülteni.
https://github.com/php/php-src/security/advisories/GHSA-6wmf-4rv5-p3rp

[4] Wiz Research PHP2Shell araştırması. 21 Ekim 2026; kök neden, gösterilen etkiler ve düzeltme çalışmaları.
https://www.wiz.io/blog/php2shell-php-rce

[5] Calif Research PHP2Shell. Birincil araştırmacı anlatımı ve bildirim zaman çizelgesi.
https://calif.io/php2shell/

[6] ikow php2shell poc tespit dizini. Topluluk materyali; ürün uyumluluğu ve kapsamı bağımsız olarak doğrulanmadı. Kural içeriği, sayısı, lisansı ve uyumluluğu uygulanmadan önce kontrol edilmelidir.
https://github.com/ikow/php2shell-poc/tree/main/detection

[7] MITRE CWE 825. Geçersiz veya ömrü dolmuş işaretçinin kullanımı için kavramsal başvuru.
https://cwe.mitre.org/data/definitions/825.html

[8] PHP oturum yapılandırma kılavuzu. session.serialize_handler ve ilgili ayarların başvuru belgesi.
https://www.php.net/manual/en/session.configuration.php

[9] watchTowr PHP2Shell FAQ. 23 Ekim güncellemeli araştırmacı yayını; 21 Ekim honeypot gözlemi ve açıklama süreci.
https://watchtowr.com/intelligence/php-cve-2026-109618-faq/

[10] AWS Security Blog MadPot gözlemi. 21 Ekim 2026; 17 Ekim tarihine uzanan girişimin geriye dönük telemetri analizi.
https://aws.amazon.com/blogs/security/amazon-threat-intelligence-identifies-php2shell-exploitation-prior-to-public-disclosure-in-madpot/

[11] Ubuntu USN 8948 1. 21 Ekim 2026; desteklenen Ubuntu paketlerindeki düzeltme sürümleri.
https://ubuntu.com/security/notices/USN-8948-1

[12] Ubuntu USN 8948 2. 21 Ekim 2026; eski Ubuntu sürümleri için Pro ve Legacy destek kapsamı.
https://ubuntu.com/security/notices/USN-8948-2

[13] CyberPanel güvenlik yanıtı ve test planı. 24 Ekim güncellemeli üretici kaydı; eşleşen OPcache ve ayrı panel runtime doğrulaması.
https://cyberpanel.net/blog/php-security-october-2026-cyberpanel-response

[14] Docker resmî PHP imajı. İmaj ailesi için başvuru; bu rapor belirli etiketlerin çekildiği iddiasını taşımaz.
https://hub.docker.com/_/php

[15] Docker bridge ağ belgesi. Ağ davranışı ve izolasyon yapılandırması için başvuru.
https://docs.docker.com/engine/network/drivers/bridge/

[16] PHP FPM yapılandırma kılavuzu. Havuz, worker ve pm.max_requests parametreleri için başvuru.
https://www.php.net/manual/en/install.fpm.configuration.php

Kullanım ve yayın kapsamı

Bu rapordaki uygulamalar ayrılmış ve yetkilendirilmiş laboratuvar ortamında yürütülür. Üçüncü taraf kaynakların kendi lisansları geçerlidir. Bu derleme için ayrıca bir açık lisans atanmamıştır.

EK A SNİPELAB ÇALIŞMA DOSYALARININ TAM KAYNAĞI

24. Paket yerleşimi ve kaynak eşlemesi

Aşağıdaki içerikler teslim edilen php2shell-lab/ dosyalarıyla aynıdır. Çalıştırmak için ZIP içindeki dosyaları kullanın; PDF üzerinden kopyalamada otomatik satır kayması veya tırnak dönüşümü oluşabilir. README.md kurulum akışını, VALIDATION.md gerçek test durumunu, SHA256SUMS aktarım bütünlüğünü açıklar. Hash listesi dijital imza değildir.

24.1 compose.yaml

```
name: php2shell-defense-lab
x-php: &php
  volumes:
    - ./app:/var/www/html:ro
    - ./config/00-lab.ini:/usr/local/etc/php/conf.d/00-lab.ini:ro
  networks: [lab]
  mem_limit: 256m
  cpus: 1.0
  pids_limit: 128
  security_opt: ["no-new-privileges:true"]
  cap_drop: [NET_RAW]
  tmpfs: ["/tmp:size=32m,mode=1777"]
services:
  apache-old:
    <<: *php
    image: ${PHP_OLD_APACHE:-php:8.5.11-apache}
    ports: ["127.0.0.1:18081:80"]
  apache-fixed:
    <<: *php
    image: ${PHP_FIXED_APACHE:-php:8.5.12-apache}
    ports: ["127.0.0.1:18082:80"]
  apache-mitigated:
    <<: *php
    image: ${PHP_OLD_APACHE:-php:8.5.11-apache}
    ports: ["127.0.0.1:18083:80"]
    volumes:
      - ./app:/var/www/html:ro
      - ./config/00-lab.ini:/usr/local/etc/php/conf.d/00-lab.ini:ro
      - ./config/99-mitigation.ini:/usr/local/etc/php/conf.d/99-mitigation.ini:ro
  fpm-old:
    <<: *php
    image: ${PHP_OLD_FPM:-php:8.5.11-fpm}
    profiles: [fpm]
  fpm-fixed:
    <<: *php
    image: ${PHP_FIXED_FPM:-php:8.5.12-fpm}
    profiles: [fpm]
  gateway:
    image: ${NGINX_IMAGE:-nginx:1.28.0-alpine}
    profiles: [fpm]
    depends_on: [fpm-old, fpm-fixed]
    ports: ["127.0.0.1:18084:8084", "127.0.0.1:18085:8085"]
    volumes:
      - ./config/nginx.conf:/etc/nginx/nginx.conf:ro
    networks: [lab]
    mem_limit: 96m
    cpus: 0.5
    pids_limit: 64
    security_opt: ["no-new-privileges:true"]
networks:
  lab:
    internal: true
```

24.2 config/00-lab.ini

```
; SnipeLab comparison baseline, not a production template.
memory_limit = 64M
post_max_size = 2M
upload_max_filesize = 1M
```

```
max_file_uploads = 2
max_input_vars = 100
session.save_path = /tmp
session.upload_progress.enabled = 1
fatal_error_backtraces = 1
log_errors = On
display_errors = Off
error_log = /proc/self/fd/2
```

24.3 config/99-mitigation.ini

```
session.upload_progress.enabled = 0
fatal_error_backtraces = 0
```

24.4 config/nginx.conf

```
events {}
http {
  access_log /dev/stdout;
  error_log /dev/stderr warn;
  client_max_body_size 2m;
  server {
    listen 8084;
    location ~ ^/(status|upload|oom)\.php$ {
      include /etc/nginx/fastcgi_params;
      fastcgi_param SCRIPT_FILENAME /var/www/html$fastcgi_script_name;
      fastcgi_pass fpm-old:9000;
    }
    location / { return 404; }
  }
  server {
    listen 8085;
    location ~ ^/(status|upload|oom)\.php$ {
      include /etc/nginx/fastcgi_params;
      fastcgi_param SCRIPT_FILENAME /var/www/html$fastcgi_script_name;
      fastcgi_pass fpm-fixed:9000;
    }
    location / { return 404; }
  }
}
```

24.5 app/status.php

```
<?php
declare(strict_types=1);
header('Content-Type: application/json');
header('Cache-Control: no-store');
function flag_value(string $name): bool {
    return filter_var(ini_get($name), FILTER_VALIDATE_BOOLEAN);
}
echo json_encode([
    'lab' => 'SnipeLab-PHP2Shell',
    'php_version' => PHP_VERSION,
    'sapi' => PHP_SAPI,
    'pid' => getmypid(),
    'ini_file' => php_ini_loaded_file(),
    'ini_scanned' => php_ini_scanned_files(),
    'upload_progress' => flag_value('session.upload_progress.enabled'),
    'fatal_backtraces' => flag_value('fatal_error_backtraces'),
    'memory_limit' => ini_get('memory_limit'),
    'opcache_version' => phpversion('Zend OPcache') ?: null
], JSON_UNESCAPED_SLASHES | JSON_THROW_ON_ERROR);
```

24.6 app/upload.php

```
<?php
declare(strict_types=1);
header('Content-Type: application/json');
header('Cache-Control: no-store');
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit;
}
```

```
$file = $_FILES['probe'] ?? null;
if (!$file || $file['error'] !== UPLOAD_ERR_OK || $file['size'] > 4096
    || !is_uploaded_file($file['tmp_name'])) {
    http_response_code(400);
    echo json_encode(['accepted' => false]);
    exit;
}
echo json_encode([
    'accepted' => true,
    'size' => $file['size'],
    'sha256' => hash_file('sha256', $file['tmp_name']),
    'persisted' => false
], JSON_THROW_ON_ERROR);
// No move_uploaded_file(): PHP removes the temporary upload at shutdown.
```

24.7 app/oom.php

```
<?php
declare(strict_types=1);
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit;
}
header('Content-Type: application/json');
ini_set('memory_limit', '16M');
error_log('LAB_BOUNDED_OOM_DEMO pid=' . getmypid());
// One fixed allocation; no size is accepted from the HTTP request.
$bounded = str_repeat('L', 24 * 1024 * 1024);
echo json_encode(['oom_observed' => false, 'allocated' => strlen($bounded)]);
```

24.8 tools/check.py

```
#!/usr/bin/env python3
"""Fixed-loopback functional controls. No claim of CVE exploitation."""
import argparse
import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path
from urllib.error import HTTPError, URLError
from urllib.request import (Request, build_opener, ProxyHandler,
                            HTTPRedirectHandler)

ROOT = Path(__file__).resolve().parents[1]
PAYLOAD = b'SnipeLab normal upload\n'
TARGETS = [
    ('apache-old', 18081, '8.5.11', 'apache2handler', True),
    ('apache-fixed', 18082, '8.5.12', 'apache2handler', True),
    ('apache-mitigated', 18083, '8.5.11', 'apache2handler', False),
    ('fpm-old', 18084, '8.5.11', 'fpm-fcgi', True),
    ('fpm-fixed', 18085, '8.5.12', 'fpm-fcgi', True),
]

class NoRedirect(HTTPRedirectHandler):
    def redirect_request(self, req, fp, code, msg, headers, newurl):
        return None

OPENER = build_opener(ProxyHandler({}), NoRedirect())

def fetch(port, path, data=None, headers=None):
    if port not in {t[1] for t in TARGETS}:
        raise ValueError('Port is not an authorized fixed lab port')
    if path not in ('/status.php', '/upload.php', '/oom.php'):
        raise ValueError('Unknown lab endpoint')
    req = Request('http://127.0.0.1:%d%s' % (port, path),
                  data=data, headers=headers or {})
    try:
        with OPENER.open(req, timeout=10) as response:
            raw = response.read(65537)
            if len(raw) > 65536:
                raise ValueError('Response exceeds evidence size limit')
            parsed = json.loads(raw)
            if not isinstance(parsed, dict):
                raise ValueError('Expected a JSON object')
            return response.status, parsed
    except HTTPError as exc:
        exc.close()
        return exc.code, {'http_error': True}

def multipart():
    boundary = 'SnipeLabBoundary01'
    body = ('--' + boundary + '\r\n'
            'Content-Disposition: form-data; name="probe"; '
            'filename="probe.txt"\r\n'
            'Content-Type: text/plain\r\n\r\n').encode()
    body += PAYLOAD + ('\r\n--' + boundary + '--\r\n').encode()
    return body, {'Content-Type': 'multipart/form-data; boundary=' + boundary}

def inspect_target(target, oom=False):
    name, port, version, sapi, progress = target
    item = {'target': name, 'checks': {}, 'scope': 'functional_baseline',
            'security_posture': ('patched_for_two_cves_only' if 'fixed' in name
                                 else 'known_affected_partial_mitigation'
                                 if 'mitigated' in name else 'known_affected_comparison')}
    try:
        code, status = fetch(port, '/status.php')
        checks = item['checks']
        checks['status_http'] = code == 200
        checks['identity'] = status.get('lab') == 'SnipeLab-PHP2Shell'
        checks['version'] = status.get('php_version') == version
        checks['sapi'] = status.get('sapi') == sapi
        checks['upload_progress'] = status.get('upload_progress') is progress
```

```
        checks['fatal_backtraces'] = status.get('fatal_backtraces') is progress
        checks['baseline_ini'] = '00-lab.ini' in str(status.get('ini_scanned'))
        if not progress:
            checks['mitigation_ini'] = '99-mitigation.ini' in str(status.get('ini_scanned'))
        item['status'] = status
        body, headers = multipart()
        code, upload = fetch(port, '/upload.php', body, headers)
        checks['normal_upload'] = (
            code == 200 and upload.get('accepted') is True
            and upload.get('size') == len(PAYLOAD)
            and upload.get('persisted') is False
            and upload.get('sha256') == hashlib.sha256(PAYLOAD).hexdigest())
        if oom:
            observation = {'scope': 'bounded_oom_observation',
                           'oom_confirmed': False,
                           'requires': 'Correlate LAB_BOUNDED_OOM_DEMO with runtime logs'}
            try:
                code, reply = fetch(port, '/oom.php', b'')
                observation.update(http_status=code, reply=reply)
            except (OSError, ValueError, URLError) as exc:
                observation['transport_error'] = str(exc)
            # A disconnected request/500 is observation, never exploit evidence.
            code, after = fetch(port, '/status.php')
            checks['healthy_after_oom_request'] = (
                code == 200 and after.get('lab') == 'SnipeLab-PHP2Shell')
            observation['after'] = after
            item['oom_observation'] = observation
    except (OSError, ValueError, URLError) as exc:
        item['error'] = str(exc)
        item['checks']['reachable_and_parseable'] = False
    item['pass'] = bool(item['checks']) and all(item['checks'].values())
    return item

def run(fpm=False, oom=False):
    targets = TARGETS if fpm else TARGETS[:3]
    results = [inspect_target(t, oom) for t in targets]
    return {'schema': 'snipelab-controls-v1',
            'time_utc': datetime.now(timezone.utc).isoformat(),
            'scope': 'version_configuration_upload_only',
            'cve_exploitation_tested': False,
            'results': results, 'pass': all(x['pass'] for x in results)}

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--fpm', action='store_true')
    parser.add_argument('--oom-demo', action='store_true')
    args = parser.parse_args()
    result = run(args.fpm, args.oom_demo)
    evidence = ROOT / 'evidence'
    evidence.mkdir(exist_ok=True)
    stamp = datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S%fZ')
    dest = evidence / ('checks-' + stamp + '.json')
    dest.write_text(json.dumps(result, ensure_ascii=False, indent=2))
    print(json.dumps({'pass': result['pass'], 'evidence': str(dest),
                      'scope': result['scope']}, ensure_ascii=False))
    return 0 if result['pass'] else 1

if __name__ == '__main__':
    raise SystemExit(main())
```

24.9 tools/collect.py

```
#!/usr/bin/env python3
"""Collect evidence and generate digest overrides from already pulled images."""
import argparse
import json
import subprocess
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
IMAGES = {'PHP_OLD_APACHE': 'php:8.5.11-apache',
          'PHP_FIXED_APACHE': 'php:8.5.12-apache',
          'PHP_OLD_FPM': 'php:8.5.11-fpm',
          'PHP_FIXED_FPM': 'php:8.5.12-fpm',
          'NGINX_IMAGE': 'nginx:1.28.0-alpine'}

def command(args):
    p = subprocess.run(args, cwd=ROOT, capture_output=True, text=True, timeout=45)
    return {'argv': args, 'returncode': p.returncode,
            'stdout': p.stdout, 'stderr': p.stderr}

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--lock', action='store_true')
    parser.add_argument('--fpm', action='store_true')
    args = parser.parse_args()
    records, locked = [], []
    selected = IMAGES if args.fpm else dict(list(IMAGES.items())[:2])
    for key, tag in selected.items():
        item = command(['docker', 'image', 'inspect', tag])
        records.append(item)
        if item['returncode'] != 0:
            raise RuntimeError('Image missing: ' + tag)
        digests = json.loads(item['stdout'])[0].get('RepoDigests', [])
        if not digests:
            raise RuntimeError('No registry digest for ' + tag)
        # Official repository expected; no arbitrary digest is manufactured.
        repo = tag.split(':')[0]
        matching = [x for x in digests if x.split('@')[0].split('/')[-1] == repo]
        if not matching:
            raise RuntimeError('No matching repository digest: ' + repo)
        locked.append(key + '=' + matching[0])
    if args.lock:
        (ROOT / 'images.lock.env').write_text('\n'.join(locked) + '\n')
    compose = ['docker', 'compose']
    if (ROOT / 'images.lock.env').exists():
        compose += ['--env-file', 'images.lock.env']
    for cmd in [['docker', 'version'], ['docker', 'compose', 'version'],
                compose + ['--profile', 'fpm', 'config'],
                compose + ['--profile', 'fpm', 'ps', '-a'],
                ['docker', 'network', 'inspect', 'php2shell-defense-lab_lab']]:
        records.append(command(cmd))
    stamp = datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S%fZ')
    directory = ROOT / 'evidence'
    directory.mkdir(exist_ok=True)
    path = directory / ('inventory-' + stamp + '.json')
    path.write_text(json.dumps({'time_utc': stamp, 'records': records}, indent=2))
    print(path)

if __name__ == '__main__':
    main()
```

24.10 tools/lifecycle_model.py

```
#!/usr/bin/env python3
"""Abstract lifecycle model, not PHP execution or real memory corruption."""
import json

def simulate(reset):
    state = {'owner_request': 1, 'reference_id': 'request-1-data'}
    live = {'request-1-data': 'temporary data'}
```

```
    if reset:
        state['reference_id'] = None
    del live['request-1-data']
    ref = state['reference_id']
    return {'reset_before_release': reset,
            'invariant_ok': ref is None or ref in live,
            'consumer_outcome': 'no stale reference' if ref is None
                                else 'invalid reference detected',
            'real_uaf_or_rce_executed': False}

if __name__ == '__main__':
    print(json.dumps({'scope': 'abstract_state_lifetime_model',
                      'without_reset': simulate(False),
                      'with_reset': simulate(True)}, indent=2))
```

24.11 tests/test_controls.py

```
"""Real local HTTP fixtures. They do not run PHP or reproduce either CVE."""
import hashlib
import importlib.util
import json
import threading
import unittest
from email import policy
from email.parser import BytesParser
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
spec = importlib.util.spec_from_file_location('check', ROOT / 'tools/check.py')
check = importlib.util.module_from_spec(spec)
spec.loader.exec_module(check)
spec2 = importlib.util.spec_from_file_location('model', ROOT / 'tools/lifecycle_model.py')
model = importlib.util.module_from_spec(spec2)
spec2.loader.exec_module(model)

class Fixture(BaseHTTPRequestHandler):
    wrong_version = False
    corrupt_upload = False
    redirect = False
    def log_message(self, *args):
        pass
    def send_json(self, payload, code=200):
        raw = json.dumps(payload).encode()
        self.send_response(code)
        self.send_header('Content-Type', 'application/json')
        self.send_header('Content-Length', str(len(raw)))
        self.end_headers()
        self.wfile.write(raw)
    def do_GET(self):
        if self.redirect:
            self.send_response(302)
            self.send_header('Location', 'http://example.invalid/no-follow')
            self.end_headers()
            return
        target = next(t for t in check.TARGETS if t[1] == self.server.server_port)
        name, port, version, sapi, progress = target
        self.send_json({'lab': 'SnipeLab-PHP2Shell',
                        'php_version': '0.0.0' if self.wrong_version else version,
                        'sapi': sapi, 'pid': port,
                        'upload_progress': progress, 'fatal_backtraces': progress,
                        'ini_scanned': '00-lab.ini;99-mitigation.ini'})
    def do_POST(self):
        body = self.rfile.read(int(self.headers.get('Content-Length', 0)))
        if self.path == '/oom.php':
            self.send_json({'fixture': True}, 500)
            return
        message = BytesParser(policy=policy.default).parsebytes(
            ('Content-Type: ' + self.headers['Content-Type'] + '\r\n\r\n').encode() + body)
        parts = list(message.iter_parts())
        data = parts[0].get_payload(decode=True)
        self.send_json({'accepted': True, 'persisted': False, 'size': len(data),
                        'sha256': 'bad' if self.corrupt_upload
```

```
                        else hashlib.sha256(data).hexdigest()})

class Controls(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.servers = []
        try:
            for _, port, *_ in check.TARGETS:
                s = ThreadingHTTPServer(('127.0.0.1', port), Fixture)
                cls.servers.append(s)
                threading.Thread(target=s.serve_forever, daemon=True).start()
        except Exception:
            cls.tearDownClass()
            raise
    @classmethod
    def tearDownClass(cls):
        for s in cls.servers:
            s.shutdown()
            s.server_close()
    def tearDown(self):
        Fixture.wrong_version = Fixture.corrupt_upload = Fixture.redirect = False
    def test_all_five_http_fixtures_pass(self):
        result = check.run(fpm=True)
        self.assertTrue(result['pass'])
        self.assertEqual(len(result['results']), 5)
        self.assertFalse(result['cve_exploitation_tested'])
    def test_wrong_version_fails(self):
        Fixture.wrong_version = True
        self.assertFalse(check.run()['pass'])
    def test_upload_corruption_fails(self):
        Fixture.corrupt_upload = True
        self.assertFalse(check.run()['pass'])
    def test_redirect_not_followed(self):
        Fixture.redirect = True
        code, _ = check.fetch(18081, '/status.php')
        self.assertEqual(code, 302)
    def test_oom_500_is_not_exploit_proof(self):
        result = check.run(oom=True)
        self.assertTrue(result['pass'])
        for item in result['results']:
            self.assertFalse(item['oom_observation']['oom_confirmed'])
    def test_unknown_target_is_rejected(self):
        with self.assertRaises(ValueError):
            check.fetch(80, '/status.php')
    def test_bad_endpoint_is_rejected(self):
        with self.assertRaises(ValueError):
            check.fetch(18081, '/other')
    def test_lifecycle_invariants(self):
        self.assertFalse(model.simulate(False)['invariant_ok'])
        self.assertTrue(model.simulate(True)['invariant_ok'])

if __name__ == '__main__':
    unittest.main()
```

25. Paket sürümleme ve kurumsal dağıtım notu

Her kurum, kendi kabul kayıtlarını `evidence/` altında ayrı zaman damgasıyla saklamalıdır. `images.lock.env`, imajlar gerçekten çekildikten sonra o ortamda üretilir; paylaşılan paket içinde uydurma veya boş digest bulunmaz. Sonraki çalıştırmalarda aynı digest ve aynı kod sürümü birlikte kullanılmalıdır.

Rapor ve paket sürümü 3.0’dır. Yerel HTTP fixture testleri gerçek PHP çalışma testlerinden ayrı tutulur. Yeni paket veya imaj seçilirse sadece etiket değiştirmek yerine beklenti sürümü, doğrulama kaydı ve rapor birlikte güncellenmelidir.